Politecnico di Torino

Master's Degree in Cybersecurity

Laboratory 5
Cloud Forensics

Author:
Stefano Pietro Sternativo
s346910

Professor:
Andrea Atzeni

Academic Year: 2025/2026

Last update: 2026-07-25

Contents

1 Purpose and Scenario

CLASSIFIED — Memorandum

Memorandum — Commandant Isabella Ricci, Polizia Postale
To: Digital Forensics Unit — All Roles
Subject: New Engagement — NovaDynamics S.p.A., Operation SILENT BUCKET

Team; three months after the ransomware incident, NovaDynamics S.p.A. completed a migration of its production workloads to a cloud provider, seeking better resilience. Yesterday, their CISO Dr. Silvia Valenti reported that a batch of confidential client contracts has surfaced for sale on an underground forum. The affected data was stored in a cloud object storage bucket used by NovaDynamics' billing service (IaaS-hosted VM) and a separate customer-facing PaaS application.

NovaDynamics has authorized limited API-level access to their cloud environment, mediated through their cloud service provider, under a new Forensic Analysis Assignment dated 18 May 2026. As in past engagements, physical access to any hardware is out of question.

I expect the team to determine what happened, when, and whether the evidence available to us is sufficient to support a judicial case, given the peculiar constraints of this environment. Coordinate explicitly, and document exactly *what you could and could not verify*.

Commandant Isabella Ricci — Polizia Postale, Digital Forensics Unit

This laboratory scope covers: the IaaS/PaaS/SaaS distinction and its impact on evidence availability; interaction with a cloud object store via API; interpretation of provider-supplied event logs and their inherent gaps; identification of the authoritative ("primary") copy among redundant/replicated objects; and the legal and trust implications of relying on cloud-provider-mediated evidence.

1.1 Investigation Objectives (from the Engagement Letter)

NovaDynamics S.p.A. formally requests that the team:

- Determine which cloud service model (IaaS/PaaS/SaaS) applies to each affected component, and what evidence is realistically obtainable from each.

- Reconstruct a timeline of the exfiltration event using the log material the cloud provider has made available.
- Identify which of the several copies/versions of the affected storage object is authoritative, and document this in the chain of custody.
- Assess the legal admissibility of cloud-provider-mediated evidence, and discuss the jurisdictional complications raised by the case.
- Deliver a short technical report addressing the above, including an explicit statement of the investigation's limitations.

1.2 What We Already Have, and What We Still Need to Find Out

In a disk or memory investigation, the evidence is a single artefact (a disk image, a memory dump) that you acquire once and then examine offline, at your own pace, with full control over the copy.
In a cloud investigation, there is no single artefact to acquire: what NovaDynamics' cloud provider gives you is a set of *exports*, such as logs, metadata listings, billing reports, each produced by a different internal system of the provider, on a different schedule, with different blind spots.
Your job is not just to *read* these exports, but to figure out *what they can and cannot tell you*, and to fill the gaps with reasoning, not with more raw disk bytes (because there are none you can access).

That is exactly why this laboratory has two distinct kinds of activity:

- **Sections 3–4 (hands-on, "live")**: you will interact directly with a *local, harmless stand-in* for the real bucket, using the same commands and the same protocol (S3 API) that a real investigator would use against the real cloud provider.
  The goal here is not to find evidence but to build an intuition for *what kind of information the API itself can give you* (object listings, metadata, hashes), so that later, when reading the real exports, you understand exactly how that information was originally produced and what its natural limitations are.
- **Sections 5–6 (analysis, "the actual case")**: you will examine the exports below, which represent what NovaDynamics' cloud provider actually handed over for this case. This is where the real investigation happens.

1.3 Evidence & Access Manifest

Artefact	Format	Description
`cloudtrail_events.json`	JSON	Management/data-plane event log export, provided by the CSP
`cloudtrail_digest.json`	JSON	Digest file accompanying the log export, used by the CSP to attest log integrity
`s3_object_versions.json`	JSON	Metadata export: versions, timestamps, region tags for the affected object
`data_transfer_metrics.json`	JSON	Independent billing/data-transfer-out report, hourly granularity
Live sandbox access	MinIO (Docker)	A local S3-compatible emulation of the bucket, for direct hands-on interaction

Table 1: Evidence and access manifest for Operation SILENT BUCKET.

The second artefact, `cloudtrail_digest.json`, deserves a note here because it is the mechanism a real cloud provider uses to attest that a log export has not been altered after it was produced. Real AWS CloudTrail digest files chain each digest to the hash of the previous one and are signed with the provider's private key, so that a recipient can verify both the content and the origin of the logs. The file supplied for this laboratory is a simplified version of that format; you will examine what it does and does not let you prove in Section 8.

• Warning

All JSON artefacts must be treated as evidence exports. Verify the integrity of each file before analysis by computing its SHA-256 hash and recording it in your chain-of-custody log.
No artefact may be modified in place; work only on copies.
Note that a hashing/query tool such as `jq` (Section 5) does not, on its own, make an analysis forensically sound. Keep this distinction in mind: it will resurface in Section 8.

2 Tool Procurement

Before the analysis begins, each role must contribute to a documented tool selection decision. Three capability areas have been identified as immediate requirements for this case.

2.1 Required Capability Areas

Area 1: Cloud API Interaction
Tools must allow listing, retrieving, and hashing objects from a cloud object store via command line, without requiring direct infrastructure access.

Area 2: Structured Log Analysis
Tools must parse and query JSON-formatted event logs, filtering by event type, timestamp, and actor.

Area 3: Local Cloud Emulation
A capability to reproduce a minimal cloud object-storage API locally, for training purposes, without incurring cost or requiring a real provider account.

2.2 Background: Object Storage and S3

Before listing the tools, it is worth clarifying two things that later sections assume you already understand.

What "S3" actually is. S3 (Simple Storage Service) is the name of Amazon's object storage product, but over time its HTTP API has become a de-facto *industry protocol*: "S3-compatible" storage is now offered by most cloud providers and by open-source software, regardless of who actually built it.
Unlike a filesystem, an object store has no directories in the traditional sense: every object is stored flat inside a *bucket*, addressed by a *key* (its name/path

string) and accompanied by *metadata* (size, content type, last-modified time, and optionally custom key-value tags). Whenever this laboratory says "interact with the bucket", it means issuing HTTP requests against this API, normally through the `aws cli`, which simply builds and sends those requests for you.

Why we interact with S3 directly at all. NovaDynamics' actual cloud provider is, of course, not accessible to us: we only have the exports listed in Section 1.4. Section 4 of this laboratory has you interact with a *local stand-in* bucket instead.
This is not redundant busywork: it is the only way, short of a real provider account, to see first-hand what the S3 API actually returns for an operation like "list objects" or "get object metadata".
That first-hand experience is what lets you later judge, in Sections 5–6, whether a given export is complete, plausible, or has gaps.

2.3 Background: Docker

Docker lets you run a piece of software (here, MinIO) inside an isolated *container*, without installing it directly onto your VM's operating system and without it interfering with anything else on the system.
You do not need to understand Docker's internals for this laboratory; you only need three operations, all of which are shown step by step in Section 3: *run* a container (start the software), *list* running containers (check it is alive), and *stop/remove* a container (clean up afterwards). We use Docker here purely as a convenient, disposable way to stand up a local S3-compatible server, *not* as a forensic subject in its own right (container forensics is a distinct topic, not covered by this laboratory).

2.4 Tool Versions and Reproducibility

Every command in Section 3 specifies an explicit version for the tools it installs or runs, rather than pulling whatever the current release happens to be.
This is deliberate, and it is a forensic requirement rather than a stylistic preference: an analysis that cannot be reproduced is an analysis that cannot be defended. Different versions of the same tool can behave differently on identical input, and the versions recorded here are the ones under which this laboratory was verified end to end.

This is not a hypothetical concern: S3-compatible tools and emulators change their licensing terms, their default request behaviour, and their supported

protocol features often enough that a laboratory written against "whatever version is current" can silently stop working, or start behaving differently, within months.
MinIO is therefore pinned below to a specific release tag. If a future release changes its licensing or its API behaviour, the pinned tag keeps this laboratory reproducible until it is formally revised.
Record in your notes the exact version of every tool you actually used, including `aws cli` (`aws --version`) and `jq` (`jq --version`), even where this text does not pin them: your report should let a third party re-run your analysis and obtain the same results.

2.5 Pre-approved Tools

`aws cli`
: Standard command-line interface for interacting with S3-compatible APIs; usable against both real endpoints and local emulators.

`jq`
: Command-line JSON processor, for querying and filtering the CloudTrail-style log export.

MinIO (Docker)
: Local, open-source, cost-free emulation of an S3-compatible object store; run inside the same CAINE/Kali VM already used in previous laboratories.
Pinned to release `RELEASE.2025-09-07T16-13-09Z`, the version under which this laboratory was verified.

`mc` (MinIO Client)
: Companion command-line client for MinIO, used in Section 4.5 to configure event notifications, a capability not exposed through the plain S3 API.

`sha256sum`
: Already used in Laboratories 2 and 3, reused here for verifying object integrity after acquisition.

3 Setting Up the Forensic Environment

This laboratory can reuse the CAINE14/Kali VM already configured for a previous forensic exercise.
For completeness, and so that this laboratory can be carried out on its own,

even if you no longer have that VM available, the full environment setup is summarised again below.

3.1 Quick VM Setup via Pre-built `.ova` Image

If you do not already have a working CAINE14/Kali VM, the fastest path is to import an official pre-built `.ova` appliance rather than installing from an ISO. This is the recommended route if you are starting from scratch for this laboratory.

1. **Download.** Get the official Kali Linux VirtualBox `.ova` image from `https://www.kali.org/get-kali/#kali-virtual-machines` (choose the VirtualBox variant). Verify the download's checksum against the value published on that same page before proceeding
   do not skip this step; it is the same integrity-verification principle used throughout this course for evidence, applied here to your own tooling.

2. **Import.** Open VirtualBox → *File* → *Import Appliance* → select the downloaded `.ova` file → *Next*.

3. **Adjust resources before finishing the import.** On the settings review screen, increase **RAM to at least 4096 MB** (the default is often lower), see the warning below for why this matters more here than in previous laboratories.

4. **Enable nested virtualisation.** After import, before first boot: select the VM → *Settings* → *System* → *Processor* tab → tick **"Enable Nested VT-x/AMD-V"**. This single setting is what allows Docker (Section 3.3) to run correctly inside a VM; without it, Docker containers may fail to start or run at very degraded performance.

5. **First boot and login.** Start the VM. Default credentials for the official Kali `.ova` are `kali` / `kali`.
   Change the password immediately after first login (`passwd`) if this VM will be kept beyond the laboratory session.

6. **Confirm networking.** Once logged in, verify the VM has a working network connection before continuing: run `ip a` (you should see an interface with an assigned IP address, not only `lo`) and then `ping -c 3 8.8.8.8` to confirm outbound connectivity.

• **Warning**

If your VirtualBox host does not expose "Enable Nested VT-x/AMD-V" at all (greyed out), your host CPU or hypervisor may not support nested virtualisation. In that case, Docker inside this VM is not guaranteed to work reliably, and you should flag this to your team.

3.2 Base Forensic VM (recap)

If you are instead reusing an existing installation rather than the quick `.ova` route above, the following recap still applies:

Choice of distribution
: CAINE 14 "Lightstream" *or* Kali Linux (Forensic Mode). Either is acceptable; both are pre-approved.

VM configuration
: RAM: 4GB minimum (3GB absolute minimum, see the warning below regarding Docker's additional memory footprint).
Network: host-only or NAT adapter, as long as outbound connectivity works (Docker needs to pull images from the internet the first time each is used).

Write-blocking
: On CAINE14, devices are read-only (`RO=1`) by default via `rbfstab`; unlock explicitly with `blockdev -setrw` only when needed. On Kali, write-blocking is *not* automatic: run `sudo blockdev -setro /dev/sdX` on any evidence device before contact.

• **Warning**

This laboratory does not involve seized physical media, so write-blocking procedures are not exercised directly here, but the same VM, once configured, is reused for the Docker/MinIO setup below. 4GB of RAM should be treated as a firm minimum, not merely a suggestion: Section 3.3 adds a containerised service on top of the base VM's own overhead, and low-RAM VMs have been observed to make the container unstable or very slow to respond.

3.3 Installing Docker in the Forensic VM

Run each of the following lines **one at a time**, waiting for each to complete before typing the next, **do not paste all four lines** as a single block: on some terminal configurations (observed on Kali's default `zsh`), pasting multi-line blocks can trigger bracketed-paste artefacts that corrupt the input and silently skip commands.

```
sudo apt update
```

```
sudo apt install docker.io
```

```
sudo systemctl enable --now docker
```

```
sudo usermod -aG docker $USER
```

- **Warning**

Stop here and restart your session. The `usermod` command above only takes effect the *next* time you log in, it does not apply retroactively to your current shell. Log out and log back in (or reboot the VM) before continuing, otherwise every subsequent `docker` command in this section will fail with a permission error unless prefixed with `sudo`.

After logging back in, confirm Docker actually works before moving on:

```
docker run hello-world
```

This should print a short "Hello from Docker!" confirmation message. If it hangs, times out, or errors out, revisit the nested-virtualisation setting from Section 3.1 before proceeding further. This is the most common cause of failure at this exact step.

Record the Docker version you are running, for the reproducibility reasons set out in Section 2.4:

```
docker --version
```

3.4 Setting Up MinIO

With Docker confirmed working, start MinIO, that is a local, open-source server that speaks the same S3 API as a real cloud provider, so that the `aws cli` commands you will run in Section 4 work identically to how they would against a real bucket.

The image tag below is pinned to the release under which this laboratory was verified, rather than `latest`, so that everyone obtains the same behaviour (Section 2.4). The `--add-host` flag is needed later, in Section 4.5, so that the container can reach a listener running on the VM itself:

```
docker run -d --name minio \
  -p 9000:9000 -p 9001:9001 \
  --add-host=host.docker.internal:host-gateway \
  -e "MINIO_ROOT_USER=minioadmin" \
  -e "MINIO_ROOT_PASSWORD=minioadmin" \
  minio/minio:RELEASE.2025-09-07T16-13-09Z server /data --console-
      address ":9001"
```

Verify the container started correctly and *stayed running* (this matters: some containers start and then immediately exit due to a configuration error, a running container is not the same as a successfully created one):

```
docker ps
```

You should see a line for `minio` with status `Up`.
If instead `docker ps` shows nothing, run `docker ps -a` (which also lists stopped/exited containers) and then inspect why it exited:

```
docker logs minio --tail 20
```

The log output should show the MinIO startup banner (version, API/console addresses) with no license or authentication error.

- **Warning**

MinIO is a *local, open-source S3-compatible emulator*, not a real cloud environment. It is used here only to give you first-hand experience with the interaction pattern (CLI → API → object store), not to reproduce the full complexity of a real provider. All log-based analysis in this laboratory relies on artefacts provided separately (Section 5), reflecting the fact that a real forensic investigator receives log exports, not raw infrastructure access.

3.5 Configuring `aws cli` Against the Local Endpoint

The `aws cli` is a generic client: the same tool talks to a real AWS account or to any S3-compatible endpoint, depending only on which credentials and endpoint URL you give it. Install it first if not already present (it ships by default on CAINE14; on Kali it must be installed explicitly):

```
sudo apt install awscli
```

Record its version too, then configure it with the MinIO root credentials. Run each line separately, exactly as in Section 3.3, do *not* paste them as one block:

```
aws --version
```

```
aws configure set aws_access_key_id minioadmin
```

```
aws configure set aws_secret_access_key minioadmin
```

```
aws configure set region eu-west-1
```

Verify the credentials were actually stored before moving on, do not assume the three commands above succeeded silently:

```
aws configure list
```

You should see non-empty (masked) values for `access_key` and `secret_key`. If either shows as empty, repeat the corresponding `aws configure set` command above.
Finally, define the alias used throughout the rest of this laboratory:

```
alias awslocal="aws --endpoint-url=http://localhost:9000"
```

- **Warning**

This `alias` only lasts for the current shell session. If you close the terminal or open a new one, you will need to redefine it (or add it to `~/.bashrc` / `~/.zshrc` if you want it to persist).

- **Warning**

An unorthodox practice, flagged as such. The two commands above put the MinIO root credentials on the command line in cleartext. This is acceptable *here* only because they are throwaway credentials for a local, disposable sandbox holding no real evidence. In any real environment it would be poor practice, and for reasons that matter forensically as much as operationally: the credentials remain visible in your shell history (`~/.zsh_history`), in the process list while the command runs (`ps aux`), and, for the values passed to `docker run`, in the container metadata retrievable by any local user with `docker`

`inspect minio`. An investigator who works this way on a live system both exposes the credentials and writes avoidable artefacts onto the machine under examination.
Two straightforward alternatives: write the credentials directly into `~/.aws/credentials` with restrictive permissions (`chmod 600`), so they never transit the command line; or, for the container, place them in an `.env` file (also `chmod 600`) and pass it with `docker run --env-file`, which keeps them out of `docker inspect` output.
Keep this in mind for Section 8: the same question, who can see and who can alter the material your conclusions rest on, applies both to the credentials you use and to the evidence you are given.

Question

`aws cli` does not distinguish between a real AWS account and a local emulator: it is the configuration, endpoint, credentials, region, that decides what it actually talks to, not the tool itself.

The credentials you just configured (`minioadmin`/`minioadmin`) grant full administrative access to this sandbox bucket. In a real investigation, you would not receive credentials like these: you would be issued access scoped specifically to the case, by the cloud provider, under the terms of the engagement.

What does the client's inability to distinguish these two situations tell you about how much trust you can place in a command's output, if you do not independently know how the person who ran it configured their access? And why does the gap between "administrative access" and "access scoped for this investigation" matter for chain of custody, specifically?

Your answer:

4 First Contact with the Bucket

The purpose of this section is *not* to find evidence, the sandbox bucket you are about to create contains nothing incriminating. The purpose is to build a first-hand, concrete understanding of what the S3 API actually gives you when you ask it for something, so that in Sections 5–6, when you read exports that someone else produced, you know exactly what process generated them and what it could or could not have captured.

4.1 Creating the Test Bucket and Uploading Objects

Run each command separately and check its output before moving to the next one:

```
awslocal s3 mb s3://novadynamics-billing-archive
```

This creates an empty bucket. “mb” stands for “make bucket”; a bucket is the top-level container for objects, roughly analogous to a top-level folder, except that S3 buckets cannot be nested inside one another.

```
echo "test content" > contract_2026_0091.pdf
```

(You are creating a harmless placeholder file locally, standing in for the real contract document, its actual content does not matter for this exercise.)

```
awslocal s3 cp contract_2026_0091.pdf s3://novadynamics-billing-
   archive/
```

This uploads (“puts”) the object into the bucket,
under the key `contract_2026_0091.pdf`.

```
awslocal s3 ls s3://novadynamics-billing-archive/
```

This lists the bucket’s contents, you should see your uploaded file, its size, and its upload timestamp.

4.2 Integrity Verification of the Retrieved Object

```
awslocal s3 cp s3://novadynamics-billing-archive/contract_2026_0091
   .pdf ./retrieved.pdf
```

```
sha256sum retrieved.pdf
```

Question

You have just computed a hash of an object retrieved through the cloud provider's own API, you did not access any underlying disk.
What exactly does this hash prove, and what does it *not* prove, compared to a hash computed on a full disk image acquired directly from a device?

Your answer:

4.3 Listing Object Metadata

```
awslocal s3api head-object --bucket novadynamics-billing-archive --
    key contract_2026_0091.pdf
```

This is the same category of operation, an object metadata lookup, that produced the `s3_object_versions.json` export you will analyse in Section 6.
Take a moment to compare the fields you see here with the fields inside that export file; they should look structurally similar, because they come from the same kind of underlying API call.

Question

Compare the metadata fields available here to what a dedicated file-metadata tool such as ExifTool can extract directly from an acquired file.
What forensically useful information is missing, and why?

Your answer:

4.4 Where Event Logs Come From

Everything you have done so far in this section left no trace that you could go back and read.
The bucket holds the object, and the object carries its metadata, but nothing so far records *that you asked for it*. That record, the event log, is a separate mechanism, and it exists only if someone turned it on beforehand.

This is the single most important structural fact about the evidence you will analyse in Section 5, so it is worth arriving at it by experiment rather than by assertion. In the next subsection you will configure event notifications on your sandbox bucket, then repeat operations you have already performed, and observe the events appear.
The point is not the configuration itself but what it implies: an event log is a *decision* taken by whoever administers the environment, before the fact, and it can be scoped narrowly, disabled, or never enabled at all.

4.5 Configuring Event Notifications

MinIO exposes event notifications through its own administrative client, `mc`, rather than through the plain S3 API. Install it first:

```
curl -L -O https://dl.min.io/client/mc/release/linux-amd64/mc
```

The `-L` flag is required: this URL currently redirects, and without it `curl` saves the small redirect response instead of the actual binary. Check the file size before proceeding, it should be tens of megabytes, not a few hundred bytes:

```
ls -la mc
```

```
chmod +x mc
```

```
sudo mv mc /usr/local/bin/
```

```
mc --version
```

Register your local MinIO instance with `mc`:

```
mc alias set local http://localhost:9000 minioadmin minioadmin
```

Events have to be delivered somewhere. We will send them to a small HTTP listener running on the VM, so that you can read them as they arrive. Create it in a separate terminal:

```
cat > webhook_listener.py << 'EOF'
from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get('Content-Length', 0))
        print(self.rfile.read(length).decode(), flush=True)
        self.send_response(200)
        self.end_headers()

HTTPServer(('0.0.0.0', 8888), Handler).serve_forever()
EOF
```

```
python3 webhook_listener.py
```

- **Warning**

This command will not print anything and will not return you to the prompt, that is expected. `serve_forever()` blocks the terminal on purpose, waiting for an actual HTTP request to arrive. Do not interrupt it: leave this terminal exactly as it is, and move to the next step in a separate terminal. Output will appear here only once the events start being generated in Section 4.5.

Leave that terminal running and return to your first one. Point MinIO at the listener and restart it so the new target is picked up:

```
mc admin config set local notify_webhook:1 endpoint="http://host.
    docker.internal:8888"
```

```
mc admin service restart local
```

- **Warning**

The hostname `host.docker.internal` is what lets the container reach the listener running on the VM. It resolves only because of the `--add-host` flag given to `docker run` in Section 3.4. If you started the container without that flag, remove it (`docker rm -f minio`) and start it again with the full command from Section 3.4.

Now subscribe the bucket to object-creation and object-access events:

```
mc event add local/novadynamics-billing-archive arn:minio:sqs::1:
    webhook --event put,get,delete
```

```
mc event ls local/novadynamics-billing-archive
```

With notifications now active, repeat two operations you already performed in Sections 4.1 and 4.2, and watch the listener terminal as each one runs:

```
awslocal s3 cp contract_2026_0091.pdf s3://novadynamics-billing-
    archive/
```

```
awslocal s3 cp s3://novadynamics-billing-archive/contract_2026_0091
    .pdf ./retrieved2.pdf
```

The upload should produce one event, `s3:ObjectCreated:Put`.
The download, however,produces two: `s3:ObjectAccessed:Head` followed immediately by `s3:ObjectAccessed:Get`.
This is not a bug in the setup: `aws s3 cp` issues a `HeadObject` call first, to check the object exists and read its size, before issuing the `GetObject` that actually retrieves it. One command you typed produced two discrete, separately timestamped events.

Each event carries a timestamp, the requesting identity, the source IP address, and the object key.
Compare its shape with the records inside `cloudtrail_events.json`: the field names differ, because that export follows AWS CloudTrail's schema rather than MinIO's, but the information carried is the same, and it is the same class of mechanism that produced it.

Question

You performed the operations in Sections 4.1–4.3 *before* configuring notifications, and repeated two of them afterwards. Only the later ones produced events.

(a) What, concretely, would remain of the earlier operations if you had to investigate this bucket now, having arrived after the fact?

(b) The subscription you created covers `put`, `get` and `delete`. Suppose an administrator had subscribed only to `put` and `delete`. The download you just repeated actually produced *two* events, a `Head` and a `Get`, both classified as object-access events. Would narrowing the subscription to only `put,delete` have removed both, one, or

neither? How would the resulting gap look to an investigator reading the export, distinguishable from an operation that never happened?

Your answer:

5 Analysis of Provided Log Material

This is where the actual investigation begins. Everything from this point onward uses only the exports listed in Section 1.4, not the sandbox from Sections 3–4, which has already served its purpose.

Install `jq` first if not already present on your VM:

```
sudo apt install jq
```

```
jq --version
```

- **Warning**

A quick note on tool soundness before you start querying: `jq`, used as below, only reads the input file and prints to your terminal, it never modifies the file in place.
This makes it compatible with forensic soundness *if you follow the right procedure*: hash each JSON file before you touch it (as required in Section 1.4), never redirect `jq`'s output back onto the original file, and work from a copy. Note, though, that this soundness comes from *how you use the tool*, not from `jq` being a certified forensic tool in the way Autopsy or Volatility are; it is a generic, general-purpose JSON processor. Keep this distinction in mind for Section 8.

5.1 Initial Survey

```
jq '.Records[] | {eventTime, eventName, sourceIPAddress,
   userIdentity: .userIdentity.userName}' cloudtrail_events.json
```

This query extracts, from every event record, only the four fields most relevant for a first pass: when it happened, what happened, from where, and by whom. Scan the output chronologically.

Question

List, in order, every event associated with the `svc-billing-sync` role on 19 May 2026. What is anomalous about the `sourceIPAddress` of the first event in that sequence, and how would you have discovered this anomaly systematically rather than by simply reading the note field provided here?

Your answer:

5.2 The Missing Data-Plane Events

Recall from lecture the distinction between *management-plane* events (actions on the cloud resources themselves: creating a bucket, changing its policy, listing its contents) and *data-plane* events (actions on the actual data inside those resources: reading or writing individual objects).
Whether data-plane events are logged at all is a separate, opt-in configuration choice on most cloud platforms, it does not come "for free" alongside management-plane logging.

Question

Between the two `PutBucketPolicy` events at `03:16:22Z` and `03:45:51Z`, no `GetObject` events appear anywhere in the log. Explain precisely *why* this is possible in a real cloud environment, referencing the distinction between management-plane and data-plane events. Is the absence of these events itself evidence, and if so, of what?

Your answer:

5.3 Cross-Validating the Finding with a Second, Independent Method

Recall from lecture that, during acquisition, using more than one hashing algorithm at once strengthens confidence in an integrity claim: if two independent computations agree, an error or manipulation in just one of them becomes far less plausible.
The same logic applies to analysis, not just acquisition: a finding that depends entirely on a single tool's correct behaviour is weaker than one confirmed by two independent tools that parse the data in different ways.

Confirm the absence of `GetObject` events from Section 5.2 using a tool that does not depend on `jq`'s JSON parser at all:

```
grep -c '"eventName": "GetObject"' cloudtrail_events.json
```

This performs a dumb, literal text search for the string, entirely independently of `jq`'s JSON interpretation. The result should be `0`, agreeing with what you observed in Section 5.2.

Question

The `grep` command above and the `jq` query in Section 5.1 use fundamentally different methods to read the same file (naive text search vs. structured JSON parsing). Why does their agreement here strengthen your finding more than simply re-running the exact same `jq` query a second time would? Can you construct a (contrived) scenario in which `jq` and `grep` would legitimately disagree on this kind of question, even with perfectly valid, unaltered JSON?

Your answer:

Since the log export itself has a blind spot exactly where we need it most, we turn to a second, independent artefact: a billing/data-transfer report. Billing systems and audit-logging systems are usually built and operated independently of one another inside a cloud provider, precisely because they serve different purposes, which is exactly what makes cross-referencing them useful here.

```
jq '.hourly_data_transfer_out_MB[]' data_transfer_metrics.json
```

Question

Using `data_transfer_metrics.json`, identify the anomalous hour and estimate, as precisely as the data allows, the order of magnitude of data transferred out of the bucket during the un-logged window.
This source was **not** disabled by the same bucket-policy manipulation that suppressed `GetObject` logging. Why not? What does this tell you about designing a logging architecture that is resilient to a single point of compromise?
And, precisely, what can you *not* conclude from this metric alone, for instance, can you tell how many distinct objects were read?

Your answer:

5.4 Reconstructing the Attacker's Actions

Before writing the timeline, look again at the two earliest events of 19 May, `AssumeRole` and `GetSessionToken`. Every event after them is attributed to the role `svc-billing-sync`, which is a legitimate service identity and would raise no suspicion on its own. These two events are where that role was taken up, and they carry a field the later ones do not foreground: `sourceIdentity`, recording which principal invoked the assumption.

```
jq '.Records[] | select(.eventName=="AssumeRole" or .eventName=="
   GetSessionToken") | {eventTime, eventName, sourceIPAddress, arn:
    .userIdentity.arn, sourceIdentity: .userIdentity.sourceIdentity
   }' cloudtrail_events.json
```

Question

Distinguish the two identities present in these records: the role under which the subsequent actions were performed, and the principal that assumed it.
(a) Had `sourceIdentity` been absent from the export, what could you still have concluded about *who* acted, and what would have become unprovable?
(b) An assumed role is a legitimate mechanism, used here by the billing service every day. What, in this specific sequence, separates the 19 May assumption from ordinary use, and would that distinction survive if the attacker had operated from an IP address already associated with the service?

Your answer:

Question

Write a short timeline (bullet list, with UTC timestamps) reconstructing the attacker's actions from first anomalous login to policy restora-

tion, explicitly marking which steps are supported by direct log evidence and which are inferred from indirect/circumstantial evidence (Section 5.4). Why is this distinction important if this timeline is later presented in court?

Your answer:

6 Redundancy and Primary Copy Identification

Cloud storage is, by default, redundant: providers routinely replicate objects across multiple physical locations, and often across regions, for durability and disaster-recovery purposes, with no action required from the customer. From a forensic standpoint this redundancy is a double-edged sword: it means the data is unlikely to be permanently lost, but it also means that "the same file" may legitimately exist in more than one place at once, at slightly different points in a replication cycle, and only one of those copies was actually the one interacted with at the time of the incident.

6.1 Background: Cross-Region Replication

Cross-Region Replication (CRR) copies objects from a bucket in one region, the *source*, to a bucket in another, the *destination*. Once enabled, it applies automatically to every subsequent write. Three of its properties matter for what follows.

It is asynchronous. A `PutObject` on the source returns as soon as the sourceregion has the object; the copy to the destination happens afterwards, typically within seconds but with no guaranteed bound. The two regions are therefore never continuously in step.

The replica's timestamp is the time of replication, not the time of

writing. The destination object records when *it* was written, which is when replication delivered it.
A replica can consequently carry a `lastModified` value later than that of the source object it was copied from, without anything having been modified. The lag is the cause, not a second write.

The content is unchanged, and says so. Replication copies bytes verbatim, so source and replica share an identical `ETag`.
Providers additionally mark the relationship explicitly:
the destination object carries a replication status of `REPLICA`, and typically a reference back to the source version it derives from.

The forensic consequence is a rule worth stating plainly, because it generalises well beyond this laboratory: *in a replicated environment, recency of timestamp does not establish which copy was acted upon.*
Two objects with the same ETag and different timestamps are one object seen at two points in a replication cycle. To identify the copy that was actually accessed, you look at what the provider states about the relationship, the replication status and the source-version reference, rather than at which timestamp happens to be larger. The same reasoning applies to any system that propagates data asynchronously, backup targets and read replicas included; CRR is simply the case you meet here.

6.2 Inspecting the Version Export

```
jq '.versions[] | {versionId, region, lastModified, etag,
   replicationStatus}' s3_object_versions.json
```

Question

Three entries are listed for `contract_2026_0091.pdf`. Which one is the primary, authoritative copy at the time of the incident, and which is a replica? Justify your answer using at least two independent fields from the export, do not rely on `lastModified` alone.

Your answer:

6.3 The Replication Trap

Question

A colleague on your team argues that the `us-east-1` entry must be the "real" exfiltrated version, since it has the most recent timestamp. Explain why this reasoning is flawed, and what evidence in the export directly refutes it.
Then consider the converse: describe a situation in which two copies of an object in different regions differ in *content*, and explain which fields in an export like this one would let you tell that case apart from ordinary replication lag.

Your answer:

6.4 Chain of Custody Documentation

Question

What would a complete chain-of-custody record need to contain for the primary copy identified above?
Which of those fields are specific to a cloud/versioned-object context, with no equivalent when the evidence is a physical disk?

Your answer:

7 Cloud Service Model Comparison

Question

NovaDynamics' billing service actually spans two components: the automated sync job (running on a dedicated IaaS virtual machine) and a customer-facing web portal (deployed on a managed PaaS platform). For *each* of the two components, answer:

(a) What categories of forensic evidence would realistically be obtainable directly, without CSP cooperation?

(b) What would require formal cooperation from the cloud service provider?

(c) If this same billing service had instead been built entirely on a third-party SaaS platform, what would change in your answers to (a) and (b)?

Your answer:

8 Legal & Trust Module

Question (a)

A replica of the exfiltrated object resides physically in `us-east-1` (United States), while NovaDynamics and its customers are based in the EU.
To which broader problem of operating in the cloud does this situation

correspond?
Situate your answer within the real regulatory landscape, at minimum the U.S. CLOUD Act, Article 48 GDPR ruling are all directly relevant here; look up what each actually says and use them to ground your argument.
Explain why cross-region replication makes this a structural condition of cloud forensics rather than an accident of this one incident.

Your answer:

Question (b)

All of the evidence in Sections 5–6 was provided to you as an export generated by the cloud provider's own tooling. You have no way to physically verify that this export was not altered before being handed to you.
The artefact set includes `cloudtrail_digest.json`, which is the provider's own answer to this problem. In production, CloudTrail emits a digest file for each interval containing the SHA-256 hash of every log file in that interval, the hash of the preceding digest, so that digests form a chain in which altering one entry invalidates every later one, and a signature over that content made with AWS's private key, verifiable by anyone holding the corresponding public key. The file supplied here reproduces the hashes and the chain reference, but carries no signature, because producing a valid one would require the provider's private key.

(a) Inspect the digest file and verify what can be verified: recompute the SHA-256 of `cloudtrail_events.json` and compare it against the recorded value. State precisely what agreement between the two establishes, and what it leaves open.

(b) Why can reliance on a provider-produced digest be challenged in court, even when the provider acted in good faith and the signature *is* present and valid? Consider who generated the material being attested, and at what point in its lifetime the attestation was applied.

(c) Based on principles already covered in this course (hashing, chain of custody, repeatability): given what you do *not* control in a cloud environment, can provider-mediated evidence like this one ever meet the same evidentiary standard as evidence you acquired directly yourself? Justify your answer.

Your answer:

Question (c)

Throughout Section 5 you used `jq`, a general-purpose JSON tool with no forensic certification, to analyse evidence exports, and in Section 5.3 you cross-validated one finding using `grep` as an independent second method. Reconcile the following: what, precisely, makes an analysis "forensically sound": the tool used, or something else? What did the cross-validation step actually add that simply re-running the same `jq` query again would not have?

Your answer:

9 Wrap-Up

Question

In half a page, summarize the peculiarities of this investigation: what evidence categories were unavailable to you here that would have been trivial to obtain in a disk- or memory-based investigation, and what new categories (if any) did the cloud environment make available that a traditional investigation could not have provided?

Your answer: